

HW03 — Linear and Logistic Regression Specification

Ramaz ML Course 2026

This document is the **contract** for HW03. It tells you exactly what to build, what each method must do, and the shape of the public API your code must match. The test suite checks this contract directly — when your code matches the spec, the tests pass.

How to use this document: Read it once end-to-end before opening `regression.py`. Then implement methods in roughly the order tests are written (unit tests first, then `fit`, then anything left). Refer back here whenever you forget a math formula or a return shape.

Contents

1	Architecture overview	2
2	Standalone helper: <code>sigmoid</code>	2
3	LinearRegression	2
3.1	<code>__init__</code>	2
3.2	<code>forward</code>	3
3.3	<code>loss</code>	3
3.4	<code>gradient</code>	3
3.5	<code>fit</code>	3
3.6	<code>predict</code>	4
4	LogisticRegression	4
4.1	<code>__init__</code>	4
4.2	<code>forward</code>	5
4.3	<code>loss</code>	5
4.4	<code>gradient</code>	5
4.5	<code>fit</code>	5
4.6	<code>predict_proba</code>	6
4.7	<code>predict</code>	6
5	Analysis (<code>analysis.py</code>)	6
5.1	<code>load_regression_data</code>	6
5.2	<code>train_linear_model</code>	7
5.3	<code>load_classification_data</code>	7
5.4	<code>train_logistic_model</code>	7
6	How to read the test file	7
7	Restrictions	8

1 Architecture overview

Both `LinearRegression` and `LogisticRegression` follow the same five-step loop:

1. `forward(X)` computes predictions $\hat{\mathbf{y}}$ from inputs X .
2. `loss(y_pred, y_true)` returns a scalar loss.
3. `gradient(X, y_true, y_pred)` returns (dw, db) .
4. Update step: $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot dw$, $b \leftarrow b - \eta \cdot db$.
5. Repeat for `epochs` iterations.

Your `fit(X, y, epochs)` method runs this loop `epochs` times and returns the list of loss values, one per epoch. Your `predict(X)` method returns the model's prediction for fresh data:

- For linear regression, `predict(X)` is the forward pass.
- For logistic regression, `predict(X)` is a hard label (0 or 1) obtained by thresholding the forward pass's probability at 0.5.

2 Standalone helper: sigmoid

You must implement one free function before either class:

```
def sigmoid(z: torch.Tensor) -> torch.Tensor
```

A non-linear activation that squashes any real number into the interval $(0, 1)$. `LogisticRegression.forward` will apply this to a linear combination to turn it into a probability.

Element-wise sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The output is always in the open interval $(0, 1)$, has the same shape as the input, and satisfies $\sigma(-z) = 1 - \sigma(z)$.

`LogisticRegression.forward` will call this — but do not call it from inside `LinearRegression.forward`.

3 LinearRegression

The required public API is described method-by-method below. Match each signature exactly — the tests import the class and call these methods on instances of it.

3.1 `__init__`

```
def __init__(self, n_features: int, lr: float = 0.01) -> None
```

Set up a fresh model. Called once, when the user does `model = LinearRegression(n_features=8)`. After this, the model has parameters initialized to zero but hasn't seen any data yet.

Initialize:

- `self.w` — a 1-D tensor of zeros with shape `(n_features,)`, dtype `torch.float32`.
- `self.b` — a scalar tensor (0-D) with value 0.0, dtype `torch.float32`.
- `self.lr` — store the learning rate as-is.

The grader reads `self.w` and `self.b` after `fit`, so use these exact attribute names.

3.2 forward

```
def forward(self, X: torch.Tensor) -> torch.Tensor
```

Step 1 of every training iteration. Take the current parameters and produce a prediction for each row of X . Also called at inference time (via `predict`) on new data.

Compute the linear prediction:

$$\hat{\mathbf{y}} = X\mathbf{w} + b$$

- Input shape: X has shape (n, d) where $d == \text{self.w.shape}[0]$.
- Output shape: $(n,)$.

3.3 loss

```
def loss(self, y_pred: torch.Tensor, y_true: torch.Tensor) -> torch.Tensor
```

Step 2 of every training iteration. Collapse the per-example prediction error into one scalar. The training loop tries to make this number smaller.

Mean squared error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

- Inputs: both shape $(n,)$.
- Output: a 0-dimensional (scalar) tensor.

3.4 gradient

```
def gradient(self, X: torch.Tensor, y_true: torch.Tensor, y_pred: torch.Tensor)
    -> tuple[torch.Tensor, torch.Tensor]
```

Step 3 of every training iteration. Figure out which direction to move each parameter to reduce the loss. The training loop will subtract a small multiple (`lr`) of these gradients from $\mathbf{w}$ and b .

Returns the gradients of the MSE loss with respect to $\mathbf{w}$ and b . Use the formula derived in lecture (no `torch.autograd`, no `.backward()`):

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{2}{n} X^\top (\hat{\mathbf{y}} - \mathbf{y}) \quad \frac{\partial L}{\partial b} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

- $d\mathbf{w}$ has shape $(d,)$.
- db is a scalar tensor.

3.5 fit

```
def fit(self, X: torch.Tensor, y: torch.Tensor, epochs: int = 200) -> list[float]
```

Run the full training loop. This is the only method a normal user will call directly — it orchestrates `forward`, `loss`, `gradient`, and the parameter update step in sequence.

Full-batch gradient descent. The algorithm:

1. Initialize an empty history list.
 2. Repeat for `epochs` iterations:
 - (a) Compute predictions from the current parameters (call your own `forward`).
 - (b) Compute the loss between predictions and targets (call your own `loss`).
 - (c) Append the loss *as a Python float* (not a tensor) to the history.
 - (d) Compute the gradients (call your own `gradient`).
 - (e) Update parameters by stepping against the gradient: $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot d\mathbf{w}$, $b \leftarrow b - \eta \cdot db$.
 3. Return the history list.
- Returns a `list[float]` of length exactly `epochs`.
 - After `fit` returns, `self.w` and `self.b` hold the trained parameters.

3.6 predict

```
def predict(self, X: torch.Tensor) -> torch.Tensor
```

Inference. Produce predictions on new data using the trained parameters. Tests call this after `fit` to evaluate model quality.

For linear regression, `predict(X)` is identical to `forward(X)`. (It exists so that `LinearRegression` and `LogisticRegression` share a uniform inference API.)

Invariants the tests will check

- For sufficiently small `lr`, `loss_history` is non-increasing.
- On data generated by $y = Xw_{\text{true}} + b_{\text{true}} + \varepsilon$ with small ε , `fit` for enough epochs recovers w_{true} and b_{true} to within tolerance.
- `forward(X)` returns shape `(n,)`; `gradient` returns `(dw, db)` with shapes `(d,)` and scalar; `predict(X)` returns shape `(n,)`.

4 LogisticRegression

The required public API is described method-by-method below. Match each signature exactly.

4.1 __init__

```
def __init__(self, n_features: int, lr: float = 0.5) -> None
```

Same as `LinearRegression.__init__` — set up a fresh model.

Same behavior as `LinearRegression.__init__`. Default `lr` is larger because the BCE gradient is on a smaller scale — see Question 3 in the writeup.

4.2 forward

```
def forward(self, X: torch.Tensor) -> torch.Tensor
```

Step 1 of every training iteration. Produce a class-1 probability for each input row.

Probability of class 1:

$$\hat{\mathbf{y}} = \sigma(X\mathbf{w} + b)$$

Use the standalone `sigmoid` you implemented in §2.

- Output shape: `(n,)`. All values lie strictly in $(0, 1)$.

4.3 loss

```
def loss(self, y_pred: torch.Tensor, y_true: torch.Tensor) -> torch.Tensor
```

Step 2 of every training iteration. Measure prediction error for a binary classification problem. BCE penalizes confident wrong predictions much more than uncertain ones.

Binary cross-entropy:

$$\text{BCE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

- `y_true` values are in $\{0, 1\}$ (encoded as floats).
- **Numerical safety:** clamp `y_pred` into $[\varepsilon, 1 - \varepsilon]$ with $\varepsilon = 10^{-7}$ before taking `log`. Otherwise $\log(0) = -\infty$ will blow up your loss.

4.4 gradient

```
def gradient(self, X: torch.Tensor, y_true: torch.Tensor, y_pred: torch.Tensor)
    -> tuple[torch.Tensor, torch.Tensor]
```

Step 3 of every training iteration. Same purpose as the linear case — decide which direction to move each parameter to reduce the loss.

Same shape as the linear case, with the factor changed:

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{1}{n} X^\top (\hat{\mathbf{y}} - \mathbf{y}) \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Why no factor of 2 here? See writeup Question 3 — the sigmoid derivative cancels something in the BCE derivative. Lesson 4 has the full derivation.

4.5 fit

```
def fit(self, X: torch.Tensor, y: torch.Tensor, epochs: int = 300) -> list[float]
```

Run the full training loop — same orchestration as `LinearRegression.fit`.

Same training loop as `LinearRegression.fit`. The defaults (`lr=0.5`, `epochs=300`) are tuned for the Iris classification dataset used in the analysis.

4.6 predict_proba

```
def predict_proba(self, X: torch.Tensor) -> torch.Tensor
```

Inference, soft version. Returns the probability of class 1 without thresholding. Useful when callers want a confidence value.

Returns the same thing as `forward(X)` — a vector of probabilities in $(0, 1)$. This method exists so callers can read soft predictions without the threshold being applied.

4.7 predict

```
def predict(self, X: torch.Tensor) -> torch.Tensor
```

Inference, hard version. Returns the predicted class label (0 or 1) for each input.

Apply the standard 0.5 threshold:

$$y_i = \begin{cases} 1 & \text{if } \hat{y}_i > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

- Output shape: $(n,)$, values in $\{0, 1\}$.
- Output dtype should match `y_true.dtype` (float32 in our datasets).

Invariants the tests will check

- `forward` outputs are strictly in $(0, 1)$.
- For sufficiently small `lr`, `loss_history` is non-increasing.
- On linearly separable data, `predict` achieves $> 95\%$ accuracy after enough epochs.
- `predict(X)` agrees with `(predict_proba(X) > 0.5).float()`.

5 Analysis (analysis.py)

`analysis.py` contains four utility functions used by the test suite and by your final `analysis.py` run that produces plots for `writeup.md`. These are utility helpers — the learning goal is the regression classes above, not data preprocessing — so the spec names the sklearn entry points directly. Match the signatures exactly.

5.1 load_regression_data

```
def load_regression_data() -> tuple[Tensor, Tensor, Tensor, Tensor]
```

Load the California Housing dataset and return train/test splits as float32 tensors.

Steps:

1. Call `sklearn.datasets.fetch_california_housing()`.
2. Split with `sklearn.model_selection.train_test_split(test_size=0.2, random_state=42)`.
3. Standardize features: fit a `StandardScaler` on the training set only, then transform both sets.

4. Convert all four arrays to `torch.Tensor` with `dtype=torch.float32`.

Return order: `(X_train, X_test, y_train, y_test)`. Shapes: `X_train` is `(n_train, 8)`, `X_test` is `(n_test, 8)`, labels are 1-D.

5.2 train_linear_model

```
def train_linear_model(X_train: Tensor, y_train: Tensor,
    lr: float = 0.01, epochs: int = 200) -> tuple[LinearRegression, list[float]]
```

Thin wrapper: construct a fresh `LinearRegression` with `n_features=X_train.shape[1]` and the given `lr`, call its `fit` method for `epochs` epochs, and return `(model, loss_history)`.

5.3 load_classification_data

```
def load_classification_data() -> tuple[Tensor, Tensor, Tensor, Tensor]
```

Load a 2-class subset of the Iris dataset (setosa vs. versicolor), keeping only the first two features so the decision boundary can be plotted in 2D.

Steps:

1. Call `sklearn.datasets.load_iris()`.
2. Filter to rows where `target < 2`; keep only the first two feature columns.
3. Split with `train_test_split(test_size=0.2, random_state=42)`.
4. Standardize features (fit on train only).
5. Convert all four arrays to `float32` torch tensors.

Labels are in `{0,1}` encoded as floats.

5.4 train_logistic_model

```
def train_logistic_model(X_train: Tensor, y_train: Tensor,
    lr: float = 0.5, epochs: int = 300) -> tuple[LogisticRegression, list[float]]
```

Same shape as `train_linear_model` — construct a fresh `LogisticRegression`, call `fit`, return `(model, loss_history)`.

6 How to read the test file

`test_regression.py` is organized in three layers — work on them in order:

`@pytest.mark.unit` — **one method at a time**

Each test class targets one piece of one class. Get these green first; they verify the math primitives independently (forward shapes, loss values, gradients matching the formula) without requiring a working training loop.

@pytest.mark.integration — fit end-to-end

Once unit tests pass, move on. Integration tests run a full `fit` on synthetic data and check that the loss decreases, that the recovered parameters are close to ground truth, and that `predict` produces sensible outputs.

@pytest.mark.analysis — analysis.py helpers

Tests for the four helper functions in `analysis.py` (data loading and training wrappers).

Run a single layer with:

```
uv run pytest -m unit           # just unit tests
uv run pytest -m integration    # just integration
uv run pytest -m analysis       # just analysis helpers
```

7 Restrictions

- **No `torch.autograd`, no `.backward()`, no `requires_grad=True` anywhere in `regression.py`.** You are computing every gradient by hand from the derivations in lecture. The `conftest.py` fails the run if it detects `autograd` usage in your file.
- **No `sklearn.linear_model`.** Same reason — implementing logistic regression yourself is the point of the assignment.